

TECHBRIDGE

Technical Guide

How to Create and Host a Web Application
using Nginx, Docker, GitHub & AWS EC2

Document Version:	2.0
Date:	January 2025
Author:	TechBridge DevOps Team
Classification:	Internal / Client Distribution
Department:	Cloud Infrastructure & DevOps

Nginx	Docker	GitHub	AWS EC2	Linux
-------	--------	--------	---------	-------

Table of Contents

1.	Introduction & Overview	3
2.	Prerequisites & Requirements	4
3.	Architecture Overview	5
4.	Step 1 — Setting Up AWS EC2 Instance	6
5.	Step 2 — Server Initial Configuration	8
6.	Step 3 — Installing Docker & Docker Compose	9
7.	Step 4 — Creating the Web Application	11
8.	Step 5 — Dockerizing the Application	13
9.	Step 6 — Configuring Nginx as Reverse Proxy	15
10.	Step 7 — GitHub Repository Setup & CI/CD	17
11.	Step 8 — Deploying to Production	19
12.	Step 9 — SSL/TLS Configuration	21
13.	Step 10 — Monitoring & Maintenance	22
14.	Troubleshooting Guide	23
15.	Quick Reference & Cheat Sheet	24

1. Introduction & Overview

This comprehensive technical guide provides a step-by-step walkthrough for creating, containerizing, and deploying a production-ready web application using modern DevOps tools and cloud infrastructure. The guide is designed for developers, system administrators, and DevOps engineers who want to establish a robust deployment pipeline.

1.1 What You Will Learn

- Setting up and configuring an AWS EC2 instance for web hosting
- Installing and configuring Docker for application containerization
- Building a sample web application (Node.js / Python Flask)
- Creating Dockerfiles and Docker Compose configurations
- Configuring Nginx as a high-performance reverse proxy and load balancer
- Setting up GitHub repository with automated CI/CD pipelines
- Implementing SSL/TLS certificates with Let's Encrypt
- Production monitoring, logging, and maintenance best practices

1.2 Technology Stack Overview

Technology	Role	Version
Nginx	Reverse Proxy / Web Server / Load Balancer	1.24+
Docker	Application Containerization Platform	24.0+
Docker Compose	Multi-container Orchestration	v2.20+
GitHub	Source Control & CI/CD (GitHub Actions)	-
AWS EC2	Cloud Compute Infrastructure	-
Ubuntu	Server Operating System	22.04 LTS
Certbot	SSL/TLS Certificate Management	2.0+

■ **Note:** This guide assumes you have basic familiarity with Linux command line, web development concepts, and have an active AWS account. All commands are tested on Ubuntu 22.04 LTS.

2. Prerequisites & Requirements

2.1 Required Accounts & Access

- **AWS Account** — With permissions to create EC2 instances, Security Groups, and Elastic IPs
- **GitHub Account** — Free or Pro plan for repository hosting and GitHub Actions
- **Domain Name** (Optional) — A registered domain for SSL configuration
- **SSH Client** — Terminal (macOS/Linux) or PuTTY (Windows)

2.2 Recommended System Specifications

Component	Minimum	Recommended
EC2 Instance Type	t2.micro (Free Tier)	t2.small or t3.small
vCPUs	1	2
RAM	1 GB	2 GB
Storage (EBS)	8 GB (gp2)	20 GB (gp3)
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Network	Default VPC	Custom VPC (production)

2.3 Security Group Configuration

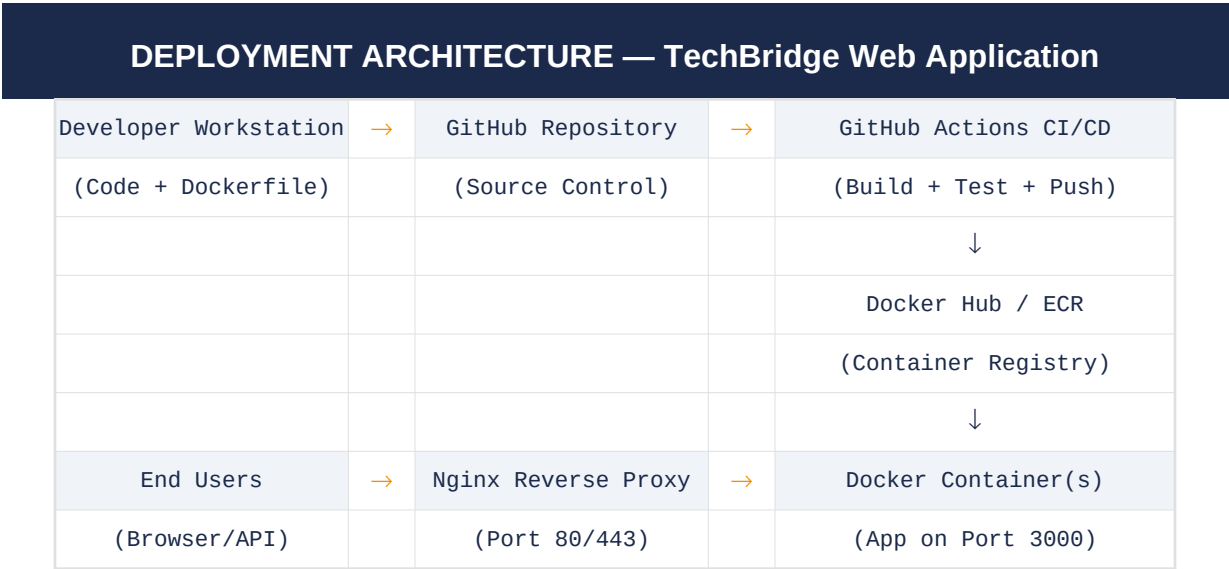
Configure the following inbound rules in your EC2 Security Group before proceeding:

Type	Protocol	Port Range	Source	Description
SSH	TCP	22	Your IP/32	SSH Access (restrict to your IP)
HTTP	TCP	80	0.0.0.0/0	Web Traffic
HTTPS	TCP	443	0.0.0.0/0	Secure Web Traffic
Custom TCP	TCP	3000	Your IP/32	App Dev Server (temporary)

■ **Warning:** Never open port 22 (SSH) to 0.0.0.0/0 in production. Always restrict SSH access to specific IP addresses. Consider using AWS Systems Manager Session Manager for SSH-less access.

3. Architecture Overview

The following architecture diagram illustrates the complete deployment pipeline and infrastructure layout that we will build throughout this guide:



3.1 Component Responsibilities

Component	Responsibility
Nginx	Acts as reverse proxy, handles SSL termination, serves static files, load balances between containers, implements security policies
Docker	Packages the application with all dependencies into a portable container, ensures consistency across development and production environments
GitHub	Hosts source code repository, manages version control, runs automated CI/CD pipelines via GitHub Actions
AWS EC2	Provides scalable cloud compute infrastructure, hosts Docker containers and Nginx, offers network security and monitoring

4. Step 1 — Setting Up AWS EC2 Instance

4.1 Launch EC2 Instance via AWS Console

Follow these steps to launch a new EC2 instance from the AWS Management Console:

- 1 Sign in to the **AWS Management Console** at <https://console.aws.amazon.com>
- 2 Navigate to **EC2 Dashboard** → Click '**Launch Instance**'
- 3 Configure the instance with the following settings:

Setting	Value
Name	techbridge-web-server
AMI	Ubuntu Server 22.04 LTS (HVM), SSD Volume Type
Instance Type	t2.micro (Free Tier) or t2.small
Key Pair	Create new: "techbridge-key" (RSA, .pem format)
Network	Default VPC, Auto-assign Public IP: Enable
Security Group	Create new: "techbridge-web-sg"
Storage	20 GB gp3, Delete on termination: Yes

4.2 Connect to Your Instance via SSH

Once the instance is running, connect using SSH:

```
# Set correct permissions for key file
$ chmod 400 techbridge-key.pem

# Connect to EC2 instance
$ ssh -i techbridge-key.pem ubuntu@<YOUR_EC2_PUBLIC_IP>

# Verify connection
ubuntu@ip-172-31-xx-xx:~$ whoami
ubuntu
```

4.3 Allocate Elastic IP (Recommended)

Assign an Elastic IP to prevent your public IP from changing after instance stop/start:

```
# Using AWS CLI
$ aws ec2 allocate-address --domain vpc

# Associate with your instance
$ aws ec2 associate-address \
  --instance-id i-0abcdef1234567890 \
  --allocation-id eipalloc-0abcdef1234567890
```

■ **Tip:** Always use an Elastic IP for production servers to ensure your DNS records remain valid even after instance restarts.

5. Step 2 — Server Initial Configuration

5.1 Update System Packages

```
# Update package index and upgrade all packages
$ sudo apt update && sudo apt upgrade -y

# Install essential utilities
$ sudo apt install -y \
    curl \
    wget \
    git \
    vim \
    htop \
    unzip \
    software-properties-common \
    apt-transport-https \
    ca-certificates \
    gnupg \
    lsb-release
```

5.2 Configure Firewall (UFW)

```
# Enable UFW firewall
$ sudo ufw allow OpenSSH
$ sudo ufw allow 80/tcp
$ sudo ufw allow 443/tcp
$ sudo ufw enable

# Verify firewall status
$ sudo ufw status verbose
Status: active
To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
```

5.3 Configure Timezone & Hostname

```
# Set timezone
$ sudo timedatectl set-timezone UTC
```



```
# Set hostname
$ sudo hostnamectl set-hostname techbridge-web

# Create a deploy user (optional but recommended)
$ sudo adduser deploy
$ sudo usermod -aG sudo deploy
$ sudo usermod -aG docker deploy
```

■■ **Warning:** Always ensure SSH access is allowed in UFW before enabling the firewall, or you will be locked out of your server!

6. Step 3 — Installing Docker & Docker Compose

6.1 Install Docker Engine

Install Docker using the official Docker repository for the latest stable version:

```
# Add Docker's official GPG key
$ sudo install -m 0755 -d /etc/apt/keyrings
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
$ sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository
$ echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker packages
$ sudo apt update
$ sudo apt install -y docker-ce docker-ce-cli \
    containerd.io docker-buildx-plugin docker-compose-plugin
```

6.2 Post-Installation Steps

```
# Add your user to the docker group
$ sudo usermod -aG docker $USER

# Apply group changes (or logout/login)
$ newgrp docker

# Verify Docker installation
$ docker --version
Docker version 24.0.7, build afdd53b

$ docker compose version
Docker Compose version v2.21.0

# Run test container
$ docker run hello-world
```

6.3 Configure Docker Daemon (Optional)

```
# Create/edit Docker daemon configuration
$ sudo vim /etc/docker/daemon.json

{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
  "storage-driver": "overlay2"
}

# Restart Docker
$ sudo systemctl restart docker
$ sudo systemctl enable docker
```

■ **Tip:** Configuring log rotation in the Docker daemon prevents log files from consuming all available disk space on your server.

7. Step 4 — Creating the Web Application

7.1 Project Structure

Create the following project structure for a Node.js web application:

```
techbridge-webapp/  
  ■■■ src/  
  ■   ■■■ index.js          # Main application entry  
  ■   ■■■ routes/  
  ■     ■■■ api.js          # API routes  
  ■     ■■■ health.js       # Health check endpoint  
  ■   ■■■ public/  
  ■     ■■■ index.html      # Landing page  
  ■     ■■■ css/  
  ■       ■■■ style.css  
  ■     ■■■ js/  
  ■       ■■■ app.js  
  ■■■ nginx/  
  ■   ■■■ nginx.conf        # Main Nginx config  
  ■   ■■■ conf.d/  
  ■     ■■■ default.conf    # Server block  
  ■■■ .github/  
  ■   ■■■ workflows/  
  ■     ■■■ deploy.yml      # CI/CD pipeline  
  ■■■ Dockerfile            # App container config  
  ■■■ docker-compose.yml    # Multi-container setup  
  ■■■ .dockerignore  
  ■■■ .gitignore  
  ■■■ package.json  
  ■■■ README.md
```

7.2 Application Code (src/index.js)

```
const express = require('express');  
const path = require('path');  
const app = express();  
  
const PORT = process.env.PORT || 3000;  
  
// Middleware  
app.use(express.json());  
app.use(express.static(path.join(__dirname, 'public')));
```

```
// Health check endpoint
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    version: process.env.APP_VERSION || '1.0.0'
  });
});

// API routes
app.get('/api/info', (req, res) => {
  res.json({
    app: 'TechBridge Web Application',
    environment: process.env.NODE_ENV || 'development'
  });
});

// Start server
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running on port ${PORT}`);
});
```

7.3 Package Configuration (package.json)

```
{
  "name": "techbridge-webapp",
  "version": "1.0.0",
  "description": "TechBridge Demo Web Application",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "test": "jest --coverage"
  },
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "jest": "^29.7.0"
  }
}
```

8. Step 5 — Dockerizing the Application

8.1 Creating the Dockerfile

Create an optimized multi-stage Dockerfile for the application:

```
# ===== Stage 1: Build =====
FROM node:20-alpine AS builder

WORKDIR /app

# Copy package files first (layer caching)
COPY package*.json ./

# Install production dependencies only
RUN npm ci --only=production && \
    npm cache clean --force

# Copy application source
COPY src/ ./src/

# ===== Stage 2: Production =====
FROM node:20-alpine AS production

# Security: Run as non-root user
RUN addgroup -g 1001 appgroup && \
    adduser -u 1001 -G appgroup -s /bin/sh -D appuser

WORKDIR /app

# Copy built artifacts from builder
COPY --from=builder --chown=appuser:appgroup /app .

USER appuser

EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider \
        http://localhost:3000/health || exit 1

CMD ["node", "src/index.js"]
```

8.2 Docker Compose Configuration

```
# docker-compose.yml
version: "3.9"

services:
  # ---- Web Application ----
  webapp:
    build:
      context: .
      dockerfile: Dockerfile
      target: production
    container_name: techbridge-app
    restart: unless-stopped
    environment:
      - NODE_ENV=production
      - PORT=3000
      - APP_VERSION=1.0.0
    networks:
      - webnet
    expose:
      - "3000"

  # ---- Nginx Reverse Proxy ----
  nginx:
    image: nginx:1.25-alpine
    container_name: techbridge-nginx
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./certbot/conf:/etc/letsencrypt:ro
      - ./certbot/www:/var/www/certbot:ro
    depends_on:
      - webapp
    networks:
      - webnet

networks:
  webnet:
    driver: bridge
```

8.3 Build and Test Locally

```
# Build the Docker image
$ docker compose build

# Start all services
$ docker compose up -d

# Check running containers
$ docker compose ps
NAME                STATUS    PORTS
techbridge-app      Up        3000/tcp
techbridge-nginx    Up        0.0.0.0:80->80/tcp

# View logs
$ docker compose logs -f webapp

# Test the application
$ curl http://localhost/health
```


9. Step 6 — Configuring Nginx as Reverse Proxy

9.1 Main Nginx Configuration (nginx/nginx.conf)

```
user  nginx;
worker_processes  auto;
error_log  /var/log/nginx/error.log warn;
pid        /var/run/nginx.pid;

events {
    worker_connections  1024;
    multi_accept on;
}

http {
    include      /etc/nginx/mime.types;
    default_type  application/octet-stream;

    # Logging format
    log_format main '$remote_addr - $remote_user '
                   '[$time_local] "$request" '
                   '$status $body_bytes_sent '
                   '"$http_referer" '
                   '"$http_user_agent"';

    access_log /var/log/nginx/access.log main;

    sendfile      on;
    tcp_nopush    on;
    tcp_nodelay    on;
    keepalive_timeout  65;
    types_hash_max_size 2048;

    # Gzip compression
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css
               application/json
               application/javascript
               text/xml application/xml;
```

```
# Security headers
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;

# Rate limiting
limit_req_zone $binary_remote_addr
    zone=api:10m rate=10r/s;

include /etc/nginx/conf.d/*.conf;
}
```

9.2 Server Block (nginx/conf.d/default.conf)

```
upstream webapp_backend {
    server webapp:3000;
    # Add more servers for load balancing:
    # server webapp2:3000;
}

server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    # Certbot challenge
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    # Proxy to application
    location / {
        proxy_pass http://webapp_backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
            $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        proxy_read_timeout 90s;
    }

    # API rate limiting
```

```
location /api/ {  
    limit_req zone=api burst=20 nodelay;  
    proxy_pass http://webapp_backend;  
}  
  
# Health check (no logging)  
location /health {  
    access_log off;  
    proxy_pass http://webapp_backend;  
}  
}
```

10. Step 7 — GitHub Repository Setup & CI/CD

10.1 Initialize Git Repository

```
# Initialize repository
$ cd techbridge-webapp
$ git init
$ git add .
$ git commit -m "Initial commit: TechBridge webapp"

# Add remote and push
$ git remote add origin \
    https://github.com/techbridge/webapp.git
$ git branch -M main
$ git push -u origin main
```

10.2 GitHub Actions CI/CD Pipeline

Create the deployment workflow at `.github/workflows/deploy.yml`:

```
name: Build and Deploy

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  REGISTRY: docker.io
  IMAGE_NAME: techbridge/webapp

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm test
```

```
build-and-push:
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4

    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKER_USER }
        password: ${ secrets.DOCKER_TOKEN }

    - name: Build and Push
      uses: docker/build-push-action@v5
      with:
        push: true
        tags: |
          ${ env.IMAGE_NAME }:latest
          ${ env.IMAGE_NAME }:${ github.sha }

deploy:
  needs: build-and-push
  runs-on: ubuntu-latest
  steps:
    - name: Deploy to EC2
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.EC2_HOST }
        username: ubuntu
        key: ${ secrets.EC2_SSH_KEY }
        script: |
          cd /opt/techbridge-webapp
          docker compose pull
          docker compose up -d --remove-orphans
          docker image prune -f
```

10.3 Required GitHub Secrets

Secret Name	Description	Where to Find
DOCKER_USER	Docker Hub username	Docker Hub account
DOCKER_TOKEN	Docker Hub access token	Docker Hub → Security → Tokens

EC2_HOST	EC2 public IP or Elastic IP	AWS EC2 Console
EC2_SSH_KEY	Private SSH key content	Your .pem key file contents

11. Step 8 — Deploying to Production

11.1 Prepare Production Server

```
# SSH into production server
$ ssh -i techbridge-key.pem ubuntu@<EC2_IP>

# Create application directory
$ sudo mkdir -p /opt/techbridge-webapp
$ sudo chown ubuntu:ubuntu /opt/techbridge-webapp

# Clone repository
$ cd /opt/techbridge-webapp
$ git clone https://github.com/techbridge/webapp.git .

# Create environment file
$ cp .env.example .env
$ vim .env
```

11.2 Launch Production Stack

```
# Pull latest images and start services
$ docker compose -f docker-compose.yml \
  -f docker-compose.prod.yml up -d

# Verify all services are running
$ docker compose ps
NAME                STATUS             PORTS
techbridge-app       Up (healthy)       3000/tcp
techbridge-nginx     Up                 0.0.0.0:80->80, 0.0.0.0:443->443

# Test the deployment
$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.25
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff

$ curl http://localhost/health
{"status":"healthy","uptime":42.5,"version":"1.0.0"}
```

11.3 Deployment Checklist

- EC2 instance running and accessible via SSH
- Security Groups configured (ports 22, 80, 443)
- Docker and Docker Compose installed
- Firewall (UFW) configured and enabled
- Application code cloned from GitHub
- Docker images built successfully
- All containers running (docker compose ps)
- Health check endpoint responding (200 OK)
- Nginx reverse proxy forwarding traffic correctly
- Application accessible via public IP or domain
- SSL/TLS certificate installed (next step)
- Monitoring and logging configured

12. Step 9 — SSL/TLS Configuration

12.1 Install Certbot & Obtain Certificate

```
# Add Certbot service to docker-compose.yml
certbot:
  image: certbot/certbot
  volumes:
    - ./certbot/conf:/etc/letsencrypt
    - ./certbot/www:/var/www/certbot
  entrypoint: "/bin/sh -c \
    'trap exit TERM; while ;; do \
    certbot renew; sleep 12h; done'"

# Obtain initial certificate
$ docker compose run --rm certbot certonly \
  --webroot --webroot-path=/var/www/certbot \
  -d yourdomain.com -d www.yourdomain.com \
  --email admin@techbridge.com \
  --agree-tos --no-eff-email
```

12.2 Update Nginx for HTTPS

```
server {
  listen 443 ssl http2;
  server_name yourdomain.com;

  ssl_certificate
    /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
  ssl_certificate_key
    /etc/letsencrypt/live/yourdomain.com/privkey.pem;

  ssl_protocols TLSv1.2 TLSv1.3;
  ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:
    ECDHE-RSA-AES128-GCM-SHA256;
  ssl_prefer_server_ciphers off;
  ssl_session_cache shared:SSL:10m;

  # HSTS
  add_header Strict-Transport-Security
    "max-age=63072000" always;

  location / {
    proxy_pass http://webapp_backend;
```

```
        # ... proxy headers ...
    }
}

# HTTP to HTTPS redirect
server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$server_name$request_uri;
}
```

13. Step 10 — Monitoring & Maintenance

13.1 Essential Monitoring Commands

```
# View container status and resource usage
$ docker compose ps
$ docker stats --no-stream

# View application logs
$ docker compose logs -f --tail=100 webapp
$ docker compose logs -f --tail=100 nginx

# Check disk usage
$ docker system df
$ df -h

# Clean up unused resources
$ docker system prune -af --volumes
$ docker image prune -af
```

13.2 Automated Health Monitoring Script

```
#!/bin/bash
# /opt/scripts/health-check.sh

HEALTH_URL="http://localhost/health"
SLACK_WEBHOOK="https://hooks.slack.com/services/..."

response=$(curl -s -o /dev/null -w "%{http_code}" $HEALTH_URL)

if [ "$response" != "200" ]; then
    echo "[ALERT] Health check failed: HTTP $response"

    # Attempt automatic recovery
    cd /opt/techbridge-webapp
    docker compose restart

    # Send alert
    curl -X POST -H "Content-type: application/json" \
        --data '{"text":"■■ TechBridge app health check failed!"}' \
        $SLACK_WEBHOOK
fi

# Add to crontab: */5 * * * * /opt/scripts/health-check.sh
```

13.3 Log Rotation

```
# /etc/logrotate.d/techbridge
/var/log/nginx/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        docker compose -f /opt/techbridge-webapp/docker-compose.yml \
            exec nginx nginx -s reload
    endscript
}
```

14. Troubleshooting Guide

Issue	Possible Cause	Solution
Container won't start	Port conflict or config error	Check logs: docker compose logs <service> Verify ports: sudo lsuf -i :80
502 Bad Gateway	App container not running or Nginx upstream misconfigured	Verify app is running: docker compose ps Check upstream name matches docker-compose service name
Permission denied	Docker socket permissions	Add user to docker group: sudo usermod -aG docker \$USER
SSL certificate errors	Certificate expired or path misconfigured	Check cert: certbot certificates Renew: certbot renew --force-renewal
High memory usage	Memory leak or insufficient instance resources	Set container memory limits: deploy.resources.limits.memory: 512M
Cannot connect via SSH	Security Group or UFW blocking	Check AWS Security Group rules Check UFW: sudo ufw status
Slow response times	No gzip, no caching, or resource constraints	Enable gzip in nginx.conf Add proxy_cache configuration Upgrade instance type

15. Quick Reference & Cheat Sheet

Docker Commands

Command	Description
<code>docker compose up -d</code>	Start all services in background
<code>docker compose down</code>	Stop and remove all containers
<code>docker compose ps</code>	List running containers
<code>docker compose logs -f</code>	Follow container logs
<code>docker compose restart <svc></code>	Restart a specific service
<code>docker compose pull</code>	Pull latest images
<code>docker compose build --no-cache</code>	Rebuild images from scratch
<code>docker system prune -af</code>	Remove all unused data
<code>docker exec -it <name> sh</code>	Open shell in container

Nginx Commands

Command	Description
<code>nginx -t</code>	Test configuration syntax
<code>nginx -s reload</code>	Reload configuration
<code>nginx -s stop</code>	Stop Nginx
<code>tail -f /var/log/nginx/error.log</code>	Monitor error logs
<code>tail -f /var/log/nginx/access.log</code>	Monitor access logs

DOCUMENT END

For questions or support, contact the TechBridge DevOps Team at devops@techbridge.com or visit our internal documentation portal.